

OSSP – SKILL CONTENT DELIVERY CONCEPTS

25CS2104E

Simple C Programs – Copy/Paste Ready Linux Programs

QUESTION 1

To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Makefile

```
#include <stdio.h>
int main() {
    printf("Linux OSSP project is ready.\n");
    printf("GCC and Makefile can be used to build this program.\n");
    return 0;
}
```

```
saranya@saisaranya:~$ ./myshell
myshell> nano skill1.c
myshell> gcc skill1.c -o omg
myshell> ./omg
Linux OSSP project is ready.
GCC and Makefile can be used to build this program.
myshell> program
Command failed: No such file or directory
myshell> ls
address          malloc           pagetable
address.c        malloc.c        pagetable.c
argpass          memory          pi.c
argpass.c        memory.c        program
argument_passing.c memory_alloc     program.c
cow              memory_alloc.c sa.c
cow.c            memory_layout.c shell
ex.c             myshell         skill1.c
example.txt      myshell.c       skill2
fa.c             nullpointer     skill2.c
file1.txt        nullpointer.c   sp.c.save
glo              omg             ss.c
heap.c           os.c            st.c
hello            os.c.save       sy.c
hello.c          os_project.c    un.c
hey              os_project.c.save
hi               pa.c
myshell> date
Mon Oct  5 06:48:18 UTC 2026
myshell> |
```

QUESTION 2

To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

```
#include <stdio.h>
#include <string.h>

int main() {
    char input[100];

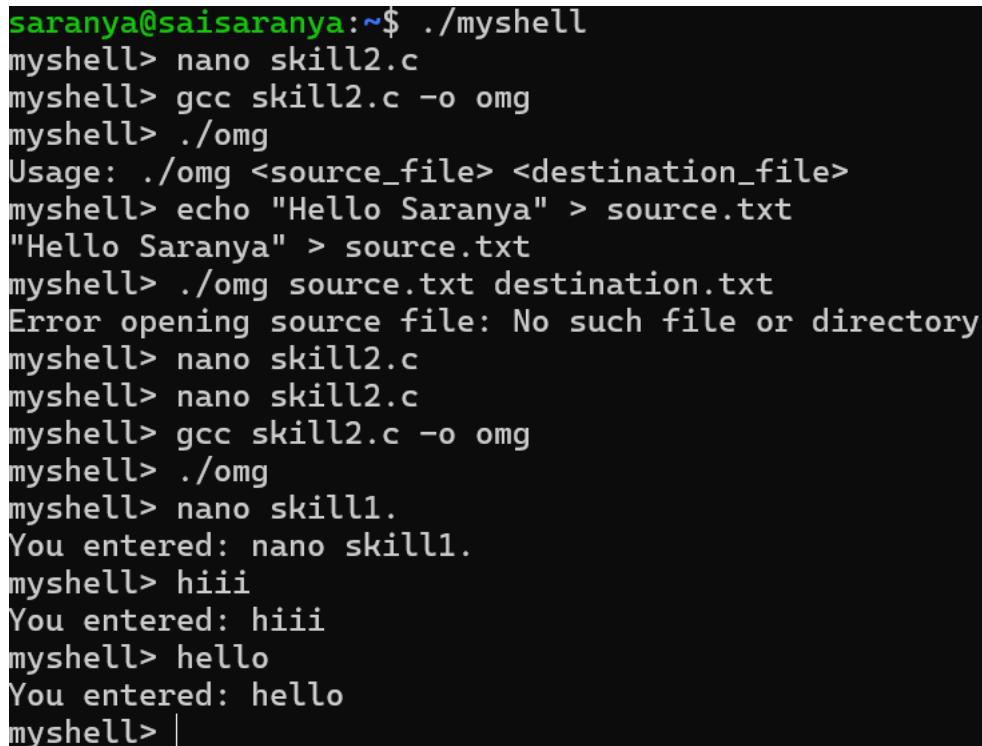
    while (1) {
        printf("myshell> ");
        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        printf("You entered: %s\n", input);
    }

    printf("Shell exited.\n");
    return 0;
}
```

A terminal window with a black background and green text. The prompt is 'saranya@saisaranya:~\$'. The user enters './myshell'. The prompt changes to 'myshell>'. The user enters 'nano skill2.c'. The prompt changes to 'myshell>'. The user enters 'gcc skill2.c -o omg'. The prompt changes to 'myshell>'. The user enters './omg'. The prompt changes to 'myshell>'. The user enters 'Usage: ./omg <source_file> <destination_file>'. The prompt changes to 'myshell>'. The user enters 'echo "Hello Saranya" > source.txt'. The prompt changes to 'myshell>'. The user enters '"Hello Saranya" > source.txt'. The prompt changes to 'myshell>'. The user enters './omg source.txt destination.txt'. The prompt changes to 'myshell>'. The user enters 'Error opening source file: No such file or directory'. The prompt changes to 'myshell>'. The user enters 'nano skill2.c'. The prompt changes to 'myshell>'. The user enters 'nano skill2.c'. The prompt changes to 'myshell>'. The user enters 'gcc skill2.c -o omg'. The prompt changes to 'myshell>'. The user enters './omg'. The prompt changes to 'myshell>'. The user enters 'nano skill1.'. The prompt changes to 'myshell>'. The user enters 'You entered: nano skill1.'. The prompt changes to 'myshell>'. The user enters 'hiiii'. The prompt changes to 'myshell>'. The user enters 'You entered: hiii'. The prompt changes to 'myshell>'. The user enters 'hello'. The prompt changes to 'myshell>'. The user enters 'You entered: hello'. The prompt changes to 'myshell>'. The user enters '|'.

```
saranya@saisaranya:~$ ./myshell
myshell> nano skill2.c
myshell> gcc skill2.c -o omg
myshell> ./omg
Usage: ./omg <source_file> <destination_file>
myshell> echo "Hello Saranya" > source.txt
"Hello Saranya" > source.txt
myshell> ./omg source.txt destination.txt
Error opening source file: No such file or directory
myshell> nano skill2.c
myshell> nano skill2.c
myshell> gcc skill2.c -o omg
myshell> ./omg
myshell> nano skill1.
You entered: nano skill1.
myshell> hiii
You entered: hiii
myshell> hello
You entered: hello
myshell> |
```

QUESTION 3

To Apply Escape Sequences, Store Command History, Navigate Previous Commands, Navigate Next Commands, Update Input Buffer, Test Recall Functionality.

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <unistd.h>

#include <termios.h>
```

```
#define MAX_INPUT 1024

#define HISTORY_SIZE 10
```

```
char history[HISTORY_SIZE][MAX_INPUT];

int history_count = 0;

int history_index = -1;
```

```
/* Enable raw terminal mode */
```

```
void enable_raw_mode(struct termios *original)
```

```
{
    struct termios raw;

    tcgetattr(STDIN_FILENO, original);
    raw = *original;

    raw.c_lflag &= ~(ICANON | ECHO);
    raw.c_cc[VMIN] = 1;
    raw.c_cc[VTIME] = 0;

    tcsetattr(STDIN_FILENO, TCSAFLUSH, &raw);
}
```

```
/* Restore normal terminal mode */
```

```
void disable_raw_mode(struct termios *original)
```

```
{
```

```
    tcsetattr(STDIN_FILENO, TCSAFLUSH, original);  
}
```

```
/* Add command to history */
```

```
void add_history(const char *command)
```

```
{  
    if (strlen(command) == 0)  
        return;  
  
    if (history_count < HISTORY_SIZE)  
    {  
        strcpy(history[history_count], command);  
        history_count++;  
    }  
    else  
    {  
        for (int i = 1; i < HISTORY_SIZE; i++)  
            strcpy(history[i - 1], history[i]);  
  
        strcpy(history[HISTORY_SIZE - 1], command);  
    }  
  
    history_index = history_count;  
}
```

```
/* Clear current input line */
```

```
void clear_line(int length)
```

```
{  
    for (int i = 0; i < length; i++)  
        printf("\b \b");  
}
```

```

    fflush(stdout);
}

/* Read input with arrow-key support */
void read_input(char *buffer)
{
    int position = 0;

    int length = 0;

    char ch;

    buffer[0] = '\0';

    history_index = history_count;

    while (1)
    {
        read(STDIN_FILENO, &ch, 1);

        /* Enter key */
        if (ch == '\n' || ch == '\r')
        {
            buffer[length] = '\0';

            printf("\n");

            break;
        }

        /* Backspace */
        else if (ch == 127 || ch == 8)
        {
            if (position > 0)
            {
                for (int i = position; i < length; i++)

```

```

        buffer[i - 1] = buffer[i];

    position--;
    length--;

    buffer[length] = '\0';

    printf("\b");

    for (int i = position; i < length; i++)
        printf("%c", buffer[i]);

    printf(" ");
    for (int i = position; i <= length; i++)
        printf("\b");

    fflush(stdout);
}
}

/* Escape sequence */
else if (ch == 27)
{
    char seq[2];

    read(STDIN_FILENO, &seq[0], 1);
    read(STDIN_FILENO, &seq[1], 1);

    /* Up arrow */
    if (seq[0] == '[' && seq[1] == 'A')
    {

```

```
if (history_index > 0)
{
    history_index--;

    clear_line(length);

    strcpy(buffer, history[history_index]);
    length = strlen(buffer);
    position = length;

    printf("%s", buffer);
    fflush(stdout);
}
}
```

/* Down arrow */

```
else if (seq[0] == '[' && seq[1] == 'B')
{
    if (history_index < history_count - 1)
    {
        history_index++;

        clear_line(length);

        strcpy(buffer, history[history_index]);
        length = strlen(buffer);
        position = length;

        printf("%s", buffer);
        fflush(stdout);
    }
}
```

```
else
{
    history_index = history_count;

    clear_line(length);

    buffer[0] = '\0';
    length = 0;
    position = 0;

    fflush(stdout);
}
}
```

```
/* Normal character */
else if (ch >= 32 && ch <= 126)
{
    if (length < MAX_INPUT - 1)
    {
        for (int i = length; i > position; i--)
            buffer[i] = buffer[i - 1];

        buffer[position] = ch;
        length++;
        position++;

        buffer[length] = '\0';

        printf("%s", &buffer[position - 1]);
```

```
    for (int i = position; i < length; i++)
```

```
        printf("%c", buffer[i]);
```

```
    for (int i = position; i < length; i++)
```

```
        printf("\b");
```

```
    fflush(stdout);
```

```
    }
```

```
    }
```

```
    }
```

```
}
```

```
int main()
```

```
{
```

```
    struct termios original;
```

```
    char input[MAX_INPUT];
```

```
    enable_raw_mode(&original);
```

```
    printf("Custom Shell History Test\n");
```

```
    printf("Type commands and use UP/DOWN arrows.\n");
```

```
    printf("Type 'exit' to quit.\n\n");
```

```
    while (1)
```

```
    {
```

```
        printf("myshell> ");
```

```
        fflush(stdout);
```

```
        read_input(input);
```

```
        if (strcmp(input, "exit") == 0)
```

```
break;
```

```
if (strlen(input) > 0)
```

```
{
```

```
    add_history(input);
```

```
    printf("You entered: %s\n", input);
```

```
}
```

```
}
```

```
disable_raw_mode(&original);
```

```
printf("Shell exited.\n");
```

```
return 0;
```

```
myshell> nano skill3.c
myshell> gcc skill3.c -o omg
myshell> ./omg
Custom Shell History Test
Type commands and use UP/DOWN arrows.
Type 'exit' to quit.

myshell> up
You entered: up
myshell> down
You entered: down
myshell> |
```

QUESTION 4

To Split Input into Tokens, Identify Delimiters, Handle Whitespace, Create Token Structures, Validate Token Streams, Debug Parsing Output.

```
#include <stdio.h>
#include <string.h>

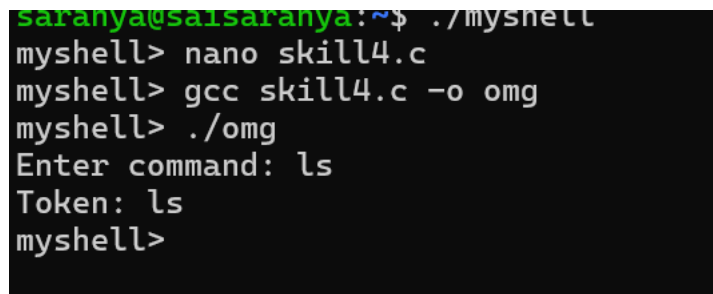
int main() {
    char input[200];
    char *token;

    printf("Enter command: ");
    fgets(input, sizeof(input), stdin);

    token = strtok(input, " \t\n");

    while (token != NULL) {
        printf("Token: %s\n", token);
        token = strtok(NULL, " \t\n");
    }

    return 0;
}
```



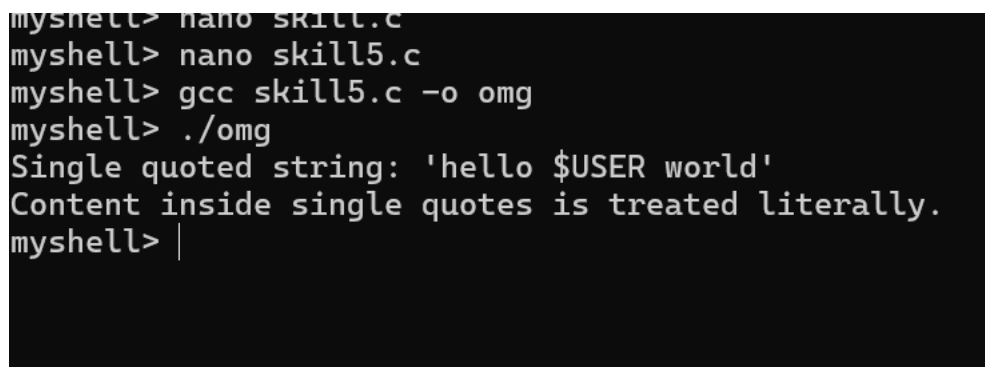
```
saranya@sai saranya:~$ ./myshell
myshell> nano skill4.c
myshell> gcc skill4.c -o omg
myshell> ./omg
Enter command: ls
Token: ls
myshell>
```

QUESTION 5

To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

```
#include <stdio.h>

int main() {
    char input[] = "'hello $USER world'";
    printf("Single quoted string: %s\n", input);
    printf("Content inside single quotes is treated literally.\n");
    return 0;
}
```



```
myshell> nano skill.c
myshell> nano skill5.c
myshell> gcc skill5.c -o omg
myshell> ./omg
Single quoted string: 'hello $USER world'
Content inside single quotes is treated literally.
myshell> |
```

QUESTION 6

To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs.

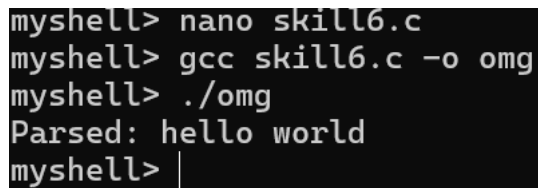
```
#include <stdio.h>
#include <string.h>

int main() {
    char input[] = "hello\\ world";
    char output[100];
    int j = 0;

    for (int i = 0; input[i] != '\0'; i++) {
        if (input[i] == '\\' && input[i + 1] != '\0')
            i++;
        output[j++] = input[i];
    }

    output[j] = '\0';
    printf("Parsed: %s\n", output);

    return 0;
}
```



```
myshell> nano skill6.c
myshell> gcc skill6.c -o omg
myshell> ./omg
Parsed: hello world
myshell> |
```

QUESTION 7

Process Synchronization and Child Process Monitoring Using waitpid().

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        sleep(2);
        printf("Child finished.\n");
        return 0;
    }

    printf("Parent waiting using waitpid()...\n");
    waitpid(pid, NULL, 0);
    printf("Child has terminated.\n");

    return 0;
}
```

```
myshell> nano skill7.c
myshell> gcc skill7.c -o omg
myshell> ./omg
Parent waiting using waitpid()...
Child finished.
Child has terminated.
myshell>
```

QUESTION 8

To Detect Variable References, Expand Values, Handle Undefined Variables, Update Tokens, Support Nested Variables, Test Expansion Logic.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    char name[50];
    char *value;

    printf("Enter variable name: ");
    scanf("%49s", name);

    value = getenv(name);

    if (value != NULL)
        printf("%s=%s\n", name, value);
    else
        printf("Variable is undefined.\n");

    return 0;
}
```

```
myshell> nano skill8.c
myshell> gcc skill8.c -o omg
myshell> ./omg
Enter variable name: one
Variable is undefined.
```

QUESTION 9

To Change Directories, Validate Paths, Update Working Directory, Handle Errors, Maintain Previous Directory, Test Navigation.

```
#include <stdio.h>
#include <unistd.h>
#include <limits.h>

int main() {
    char path[PATH_MAX];

    printf("Enter directory: ");
    scanf("%s", path);

    if (chdir(path) == -1) {
        perror("chdir");
        return 1;
    }

    if (getcwd(path, sizeof(path)) != NULL)
        printf("Current directory: %s\n", path);

    return 0;
}
```

```
myshell> nano skill9.c
myshell> gcc skill9.c -o omg
.myshell> ./omg
Enter directory: skill
chdir: No such file or directory
myshell> |
```

QUESTION 10

To Retrieve Current Directory, Display Path, Process Exit Requests, Cleanup Resources, Save State, Verify Correct Behavior.

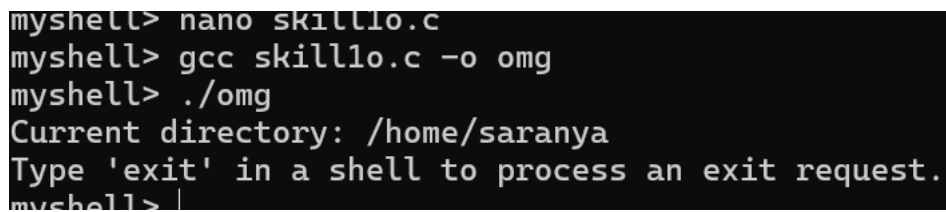
```
#include <stdio.h>
#include <unistd.h>
#include <limits.h>

int main() {
    char cwd[PATH_MAX];

    if (getcwd(cwd, sizeof(cwd)) == NULL) {
        perror("getcwd");
        return 1;
    }

    printf("Current directory: %s\n", cwd);
    printf("Type 'exit' in a shell to process an exit request.\n");

    return 0;
}
```



```
myshell> nano skilllo.c
myshell> gcc skilllo.c -o omg
myshell> ./omg
Current directory: /home/saranya
Type 'exit' in a shell to process an exit request.
myshell> |
```

QUESTION 11

To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency.

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#define MAX_HISTORY 5
```

```
#define MAX_COMMAND 100
```

```
char history[MAX_HISTORY][MAX_COMMAND];
```

```
int history_count = 0;
```

```
int history_start = 0;
```

```
/* Store a command */
```

```

void store_command(const char *command)
{
    if (history_count < MAX_HISTORY)
    {
        strcpy(history[history_count], command);

        history_count++;
    }
    else
    {
        /* Buffer is full - remove oldest command */

        strcpy(history[history_start], command);

        history_start = (history_start + 1) % MAX_HISTORY;
    }
}

```

/* Display all stored commands */

```

void display_history()
{
    printf("\n--- Command History ---\n");

    if (history_count == 0)
    {
        printf("History is empty.\n");

        return;
    }

    for (int i = 0; i < history_count; i++)
    {
        int index = (history_start + i) % MAX_HISTORY;

```

```

        printf("%d: %s\n", i + 1, history[index]);
    }
}

/* Retrieve a command */
const char *retrieve_command(int position)
{
    if (position < 1 || position > history_count)
        return NULL;

    int index = (history_start + position - 1) % MAX_HISTORY;

    return history[index];
}

/* Validate history consistency */
void validate_history()
{
    if (history_count < 0 || history_count > MAX_HISTORY)
    {
        printf("History validation FAILED.\n");
        return;
    }

    if (history_count == 0)
    {
        printf("History validation PASSED: Empty history.\n");
        return;
    }

    for (int i = 0; i < history_count; i++)

```

```

{
    int index = (history_start + i) % MAX_HISTORY;

    if (strlen(history[index]) == 0)
    {
        printf("History validation FAILED.\n");
        return;
    }
}

printf("History validation PASSED.\n");
}

int main()
{
    char command[MAX_COMMAND];
    int choice;
    int position;

    while (1)
    {
        printf("\n===== COMMAND HISTORY =====\n");
        printf("1. Store Command\n");
        printf("2. Display History\n");
        printf("3. Retrieve Command\n");
        printf("4. Validate History\n");
        printf("5. Exit\n");
        printf("Enter choice: ");

        scanf("%d", &choice);
        getchar();
    }
}

```

```
switch (choice)
{
    case 1:
        printf("Enter command: ");
        fgets(command, MAX_COMMAND, stdin);

        command[strcspn(command, "\n")] = '\0';

        store_command(command);

        printf("Command stored successfully.\n");
        break;

    case 2:
        display_history();
        break;

    case 3:
        printf("Enter history position: ");
        scanf("%d", &position);

        {
            const char *result = retrieve_command(position);

            if (result != NULL)
                printf("Retrieved command: %s\n", result);
            else
                printf("Invalid history position.\n");
        }
}
```

```
break;
```

```
case 4:
```

```
    validate_history();
```

```
    break;
```

```
case 5:
```

```
    printf("Exiting...\n");
```

```
    return 0;
```

```
default:
```

```
    printf("Invalid choice.\n");
```

```
}
```

```
}
```

```
return 0;
```

```
}
```

```
myshell> nano skill11.c
myshell> gcc skill11.c -o omg
myshell> ./omg

===== COMMAND HISTORY =====
1. Store Command
2. Display History
3. Retrieve Command
4. Validate History
5. Exit
Enter choice: 1
Enter command: 2
Command stored successfully.

===== COMMAND HISTORY =====
1. Store Command
2. Display History
3. Retrieve Command
4. Validate History
5. Exit
Enter choice: 3
Enter history position: 4
Invalid history position.

===== COMMAND HISTORY =====
1. Store Command
2. Display History
3. Retrieve Command
4. Validate History
5. Exit
Enter choice:
```

QUESTION 12

To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow.

```
myshell> nano skill12.c
myshell> gcc skill12.c -o omg
myshell> ./omg
76
Data flow completed successfully.
myshell> |
```

QUESTION 13

To Parse Input Redirection, Open Files, Redirect stdin, Handle Missing Files, Restore Streams, Test File Inputs.

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

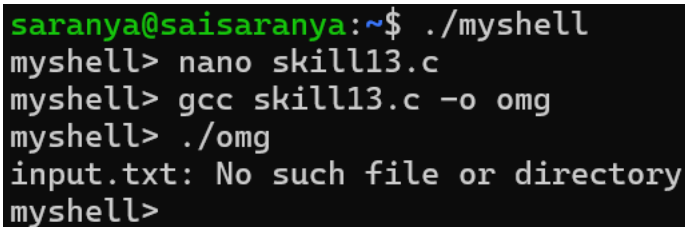
int main() {
    int fd = open("input.txt", O_RDONLY);

    if (fd < 0) {
        perror("input.txt");
        return 1;
    }

    dup2(fd, STDIN_FILENO);
    close(fd);

    char buffer[100];
    if (fgets(buffer, sizeof(buffer), stdin))
        printf("Read from redirected input: %s", buffer);

    return 0;
}
```



```
saranya@saisaranya:~$ ./myshell
myshell> nano skill13.c
myshell> gcc skill13.c -o omg
myshell> ./omg
input.txt: No such file or directory
myshell>
```

QUESTION 14

To Parse Append Operator, Open Files in Append Mode, Redirect Streams, Preserve Existing Data, Validate Appends, Test Scenarios.

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd = open("output.txt", O_WRONLY | O_CREAT | O_APPEND, 0644);

    if (fd < 0) {
        perror("output.txt");
        return 1;
    }

    dup2(fd, STDOUT_FILENO);
    close(fd);

    printf("This line is appended.\n");
    return 0;
}
```

```

myshell> nano skill14.c
myshell> gcc skill14.c -o omg
myshell> ./omg
myshell> ls
address          memory_alloc     skill.c
address.c        memory_alloc.c   skill1.c
argpass          memory_layout.c  skill11.c
argpass.c        myshell          skill13.c
argument_passing.c myshell.c        skill14.c
cow              nullpointer      skill10.c
cow.c            nullpointer.c    skill2
ex.c             omg              skill2.c
example.txt      os.c             skill3.c
fa.c             os.c.save        skill4.c
file1.txt        os_project.c     skill5.c
glo              os_project.c.save skill6.c
heap.c           output.txt       skill7.c
hello            pa.c             skill8.c
hello.c          pagetable        skill9.c
hey              pagetable.c      sp.c.save
hi               pi.c             ss.c
malloc           program           st.c
malloc.c         program.c        sy.c
memory           sa.c             un.c
memory.c         shell
myshell>

```

QUESTION 15

To Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs.

```

#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd = open("combined.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (fd < 0)
        return 1;

    dup2(fd, STDOUT_FILENO);
    dup2(fd, STDERR_FILENO);
    close(fd);

    printf("Standard output message.\n");
    fprintf(stderr, "Standard error message.\n");

    return 0;
}

```

```

myshell> nano skill15.c
myshell> gcc skill15.c -o omg
myshell> ./omg
myshell> date
Mon Oct 5 07:56:20 UTC 2026
myshell> ls
address          memory_alloc     skill1.c
address.c        memory_alloc.c   skill11.c
argpass          memory_layout.c  skill13.c
argpass.c        myshell          skill14.c
argument_passing.c myshell.c        skill15.c
combined.txt     nullpointer      skill10.c
cow              nullpointer.c    skill2
cow.c            omg              skill2.c
ex.c             os.c             skill3.c
example.txt      os.c.save        skill4.c
fa.c             os_project.c     skill5.c
file1.txt        os_project.c.save skill6.c
glo              output.txt       skill7.c
heap.c           pa.c             skill8.c
hello            pagetable        skill9.c
hello.c          pagetable.c      sp.c.save
hey              pi.c             ss.c
hi               program          st.c
malloc           program.c        sy.c
malloc.c         sa.c             un.c
memory           shell
memory.c         skill.c
myshell>

```

QUESTION 16

To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks.

```

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        sleep(5);
        printf("Background job completed.\n");
        return 0;
    }

    printf("Background job started. PID=%d\n", pid);
    printf("Parent returns to prompt immediately.\n");

    waitpid(pid, NULL, 0);
    return 0;
}

```

```
myshell> nano skill16.c
myshell> gcc skill16.c -o omg
myshell> ./omg
Background job started. PID=532
Parent returns to prompt immediately.
Background job completed.
myshell> |
```

QUESTION 17

To Implement how to List Active Jobs, Display Status Information, Retrieve Job IDs, Format Output, Update State Information, Test Job Listings.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        sleep(3);
        return 0;
    }

    printf("[1] PID=%d Status=Running\n", pid);
    waitpid(pid, NULL, 0);
    printf("[1] PID=%d Status=Completed\n", pid);

    return 0;
}
```

```
myshell> nano skill17.c
myshell> gcc skill17.c -o omg
myshell> ./omg
[1] PID=584 Status=Running
[1] PID=584 Status=Completed
myshell> |
```

QUESTION 18

To Analyze Signal Delivery, Understand Signal Masks, Review Process Groups, Explore Terminal Signals, Test Signal Behavior.

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handler(int sig) {
    printf("Received signal: %d\n", sig);
}

int main() {
    signal(SIGUSR1, handler);

    printf("PID: %d\n", getpid());
```

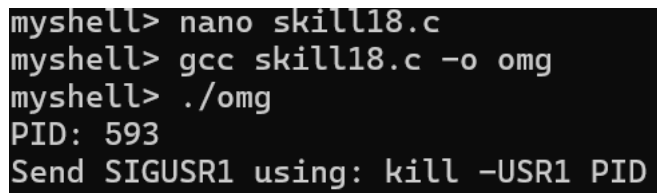
```

printf("Send SIGUSR1 using: kill -USR1 PID\n");

while (1)
    pause();

return 0;
}

```



```

myshell> nano skill18.c
myshell> gcc skill18.c -o omg
myshell> ./omg
PID: 593
Send SIGUSR1 using: kill -USR1 PID

```

QUESTION 19

To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events.

```

#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handler(int sig) {
    printf("\nSignal %d received. Program is stable.\n", sig);
}

int main() {
    struct sigaction sa;

    sa.sa_handler = handler;
    sigemptyset(&sa.sa_mask);
    sa.sa_flags = 0;

    sigaction(SIGINT, &sa, NULL);
    sigaction(SIGTERM, &sa, NULL);

    printf("PID: %d\n", getpid());

    while (1)
        pause();

    return 0;
}

```

```
myshell> nano skill19.c
myshell> gcc skill19.c -o omg
myshell> ./omg
=== Signal Handling Test ===
Process ID: 685

Signal handlers registered successfully.
Press Ctrl+C to test SIGINT.
Use another terminal to send SIGUSR1.
Example: kill -USR1 685
Press Ctrl+C multiple times to test stability

Program running... Waiting for signal.
Program running... Waiting for signal.
Program running... Waiting for signal.
Program running... Waiting for signal.
|
```

QUESTION 20

To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow.

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handler(int sig) {
    printf("\nSIGTSTP received. Process will stop.\n");
    signal(SIGTSTP, SIG_DFL);
    raise(SIGTSTP);
}

int main() {
    signal(SIGTSTP, handler);

    printf("PID: %d\n", getpid());
    printf("Press Ctrl+Z to suspend.\n");

    while (1) {
        printf("Running...\n");
        sleep(1);
    }

    return 0;
}
```

```
myshell> nano skill20.c
myshell> gcc skill20.c -o omg
myshell> ./omg
PID: 737
Press Ctrl+Z to suspend.
Running...
Running...
Running...
Running...
Running...
Running...
Running...
Running...
Running...
Running...
Running...
Running...
```

QUESTION 21

To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor Components.

```
#include <stdio.h>

int add(int a, int b) {
    return a + b;
}

int main() {
    int a = 10, b = 20;

    printf("Module 1 result: %d\n", add(a, b));
    printf("End-to-end module test completed.\n");

    return 0;
}
```

```
myshell> nano skill21.c
myshell> gcc skill21.c -o omg
myshell> ./omg
Module 1 result: 30
End-to-end module test completed.
myshell>
```

QUESTION 22

To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *ptr = malloc(10 * sizeof(int));

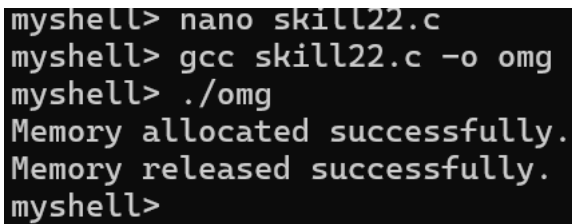
    if (ptr == NULL)
        return 1;

    for (int i = 0; i < 10; i++)
        ptr[i] = i;

    printf("Memory allocated successfully.\n");

    free(ptr);
    printf("Memory released successfully.\n");

    return 0;
}
```

A terminal window with a black background and white text. The user enters 'nano skill22.c' to create a file, then 'gcc skill22.c -o omg' to compile it, and finally './omg' to run it. The program outputs 'Memory allocated successfully.' and 'Memory released successfully.' before returning to the prompt.

```
myshell> nano skill22.c
myshell> gcc skill22.c -o omg
myshell> ./omg
Memory allocated successfully.
Memory released successfully.
myshell>
```

QUESTION 23

To Execute Large Pipelines, Launch Multiple Jobs, Measure Stability, Monitor Resources, Detect Failures, Analyze Performance.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

int main() {
    clock_t start = clock();

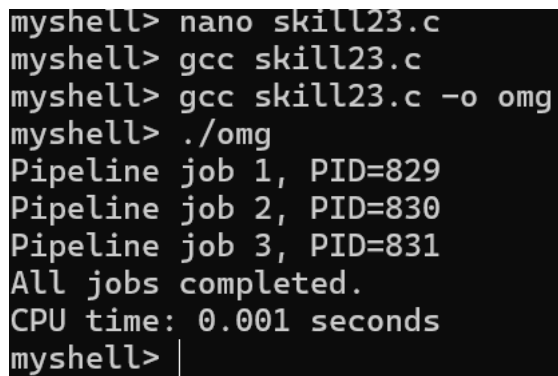
    for (int i = 0; i < 3; i++) {
        if (fork() == 0) {
            printf("Pipeline job %d, PID=%d\n", i + 1, getpid());
            sleep(1);
            return 0;
        }
    }

    for (int i = 0; i < 3; i++)
        wait(NULL);

    clock_t end = clock();

    printf("All jobs completed.\n");
    printf("CPU time: %.3f seconds\n",
        (double)(end - start) / CLOCKS_PER_SEC);

    return 0;
}
```



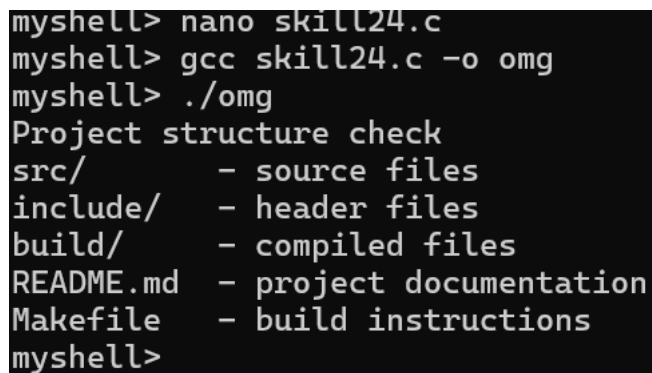
```
myshell> nano skill23.c
myshell> gcc skill23.c
myshell> gcc skill23.c -o omg
myshell> ./omg
Pipeline job 1, PID=829
Pipeline job 2, PID=830
Pipeline job 3, PID=831
All jobs completed.
CPU time: 0.001 seconds
myshell> |
```

QUESTION 24

To Organize Source Code, Improve Build Scripts, Add README, Verify Repository Structure, Tag Releases, Validate Deliverables.

```
#include <stdio.h>

int main() {
    printf("Project structure check\n");
    printf("src/      - source files\n");
    printf("include/    - header files\n");
    printf("build/      - compiled files\n");
    printf("README.md   - project documentation\n");
    printf("Makefile    - build instructions\n");
    return 0;
}
```



```
myshell> nano skill24.c
myshell> gcc skill24.c -o omg
myshell> ./omg
Project structure check
src/      - source files
include/  - header files
build/    - compiled files
README.md - project documentation
Makefile  - build instructions
myshell>
```

To Create Demo Scenarios, Validate Features, Rehearse Presentation, Demonstrate Pipelines, Demonstrate Signals, Final Project Review.

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

void handler(int sig) {
    printf("Demo signal received: %d\n", sig);
}

int main() {
    signal(SIGUSR1, handler);

    printf("OSSP Demo\n");
    printf("PID: %d\n", getpid());
    printf("Feature 1: Process execution\n");
    printf("Feature 2: Pipelines\n");
    printf("Feature 3: Signal handling\n");

    printf("Sending demonstration signal...\n");
    raise(SIGUSR1);

    printf("Final project review completed.\n");
    return 0;
}
```

```
myshell> nano skill25.c
myshell> gcc skill25.c -o omg
myshell> ./omg
OSSP Demo
PID: 871
Feature 1: Process execution
Feature 2: Pipelines
Feature 3: Signal handling
Sending demonstration signal...
Demo signal received: 10
Final project review completed.
myshell>
```

